Contents

Neuromorphic Computing for Decentralised Federated and Constrained Edge AI devices

Teoh Kai Shun
Department Of Computing
Imperial College London
London, UK
CID: 01878056

Eiman Kanjo
Department Of Computing
Imperial College London
London, UK

***Abstract*—Typical classical computing AI models' success largely lies in the vast amount of computation made possible by the advancement in chip design. However, most devices do not possess such vast computation power, specifically edge devices like cameras and sensors. In this paper, we discuss the effectiveness of utilising Neuromorphic Computing in Decentralised Federated and Constrained Edge AI devices. The paper starts by informing the reader about the motivation and reasoning for using Neuromorphic Computing in this particular setting. It then reviews the current applications of Decentralised Federated and Constrained Edge AI, discussing the limitations faced by Classical Computing, so that readers are familiar with the context. Current Implementations of Neuromorphic Computing in AI are then discussed in detail, and this is followed by a simple experiment to compare the performance of Neuromorphic Computing against Classical Computing. With the results, the paper makes a judgement on the effectiveness of Neuromorphic Computing in Constrained Edge AI devices and concludes with potential future work.**

1. Introduction

Constrained Edge AI devices usually have to work with limited computing resources, which severely limits their ability to scale with the advancement of software. However, Neuromorphic Computing may be the solution to overcome this difficulty. Neuromorphic Computing is a new school of thought for computer science that aims to reproduce computation done by the brain, specifically neurons and synapses. The brain is thought to be one of the most efficient computing units, and in this review of the literature, we wish to explore how the current state of Neuromorphic Computing can be leveraged in Edge AI devices, specifically in a decentralised, federated, and constrained setting.

2. Background Knowledge

2.1. Neuromorphic Computing

Neuromorphic Computing is a framework of computation inspired by the workings of the brain, specifically the neurones and synapses [1]. Neuromorphic Computers can be abstracted as a distributed system comprising a pool of individual neurons, which can be abstracted as spiking functions. Any two neurons can be connected, forming a synapse. With a connection (assuming it is bi-directional), two neurons can interact with each other and effect a change in each other's states. Effectively, the connections of neurons can be abstracted in a connectivity matrix. With a certain configuration and population number, the interactions of neurons in the distributed system allow the system to reflect complex behaviours. With a suitable training method, it is possible to train a Neuromorphic Computer to output a certain desired behaviour.

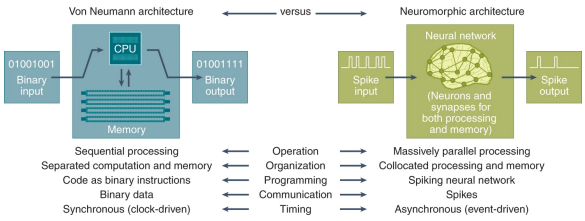

Figure 1. Figure summarises the differences of Neuromorphic devices and Classical devices. The figure is taken from (41)

However, developments in classical computers do not translate into neuromorphic computers. Thus, there is a need to develop software and hardware for Neuromorphic Computers, such that they are compatible with the Neuromorphic Computing paradigm. Furthermore, with the advancement in Neural network technology, many neural network accelerators are arguably misclassified as neuromorphic AI systems. [1], [2] releases a list of neuromorphic properties to better characterise neuromorphic computers. Below, I list and elaborate on their practicalities or significance in the context of Constrained Edge AI devices, including real-life implementations whenever possible.

2.1.1. Connectionism. Connectionism refers to the emergence of intellect from a pool of connected elements. In the case of Neuromorphic Computers, intelligence emerges from the interactions of interconnected neurons. These interactions manifest as

temporal patterns (or oscillations) of firing between modules of neurons, and connectivity configuration between the neurons significantly impacts the patterns and formation of modules. To elaborate, connectivity can affect the formation of functional neural modules by enabling synchronization or exclusion among neurons, and it affects temporal patterns by enabling synchronisation or competition between functional neuronal modules.

In the context of Constrained Edge AI devices, this can be very advantageous as connectionism can offer a more energy-efficient and computationally cheap alternative to the current implementation of Neural Networks while maintaining a satisfactory level of performance. This is so as current Neural Networks, though inspired by connectionism [2], do not fully realise its potential, as the concept of sophistication from connectionism manifests as a multiplication of parameters that grows exponentially with the size of the network and needs to be stored in persistent memory. In the Neuromorphic Computing Paradigm, such overhead cost does not exist, as sophistication from connectionism manifests as temporal patterns of firing between neurons, which requires the storing of states and connections of individual neurons. In the majority of Neuromorphic Chips, the states and connections are physical states of the chip and, as such, do not incur persistent memory overhead. An example of this is the IBM TrueNorth chip, which features configurable synaptic connections for its Digital neurons grouped into 4096 cores [3]. These synaptic connections can be tuned physically for specific tasks.

2.1.2. Parallelism. Parallelism [2] refers to the ability to perform tasks simultaneously. In Neuromorphic Computers, computation is distributed, and information is processed in parallel. This eliminates the need for a central information processing unit, overcoming the von Neumann bottleneck and leading to quick response time and low latency. In a Loihi 1 chip, 128 cores simulate 1 million neurons and 120 million synapses (connections) in parallel [4]; in SpiNNaker, 1 million ARM cores model billions of neurons asynchronously [5].

For decentralised constrained Edge AI devices, which have limited computation power, this can be significant to their performance. Parallelism enables these devices to off-load heavy computation to the cloud while maintaining functional integrity. This can be challenging for Classical computers since they risk memory corruption and overwrites between threads when parallelising. Since neuromorphic computers can leverage the vast computation available in the cloud better, they will be able to run more complex models, allowing edge devices to be more sophisticated, which can improve user experience.

2.1.3. Asynchrony. Asynchrony [2] is the absence of a global clock that synchronises all computations. The global clock is present in classical computers, which results in difficulty in parallelism in these computers due to the sequential constraint applied to computations by the global clock. In Neuromorphic Computers, however, computations are event-driven, meaning that they only fire when needed. To elaborate, at each timestep, incoming spikes from neighbouring neurons and the current state of the neuron determine whether it fires or not. For example, in BrainScaleS [6], analogue neurons process spikes asynchronously at 10000 times the speed of biological neurons, and in DVS cameras, where event-based vision sensors are paired with Akida [7] for low-power object tracking.

This gives 2 advantages in the context of Constrained Edge AI devices. Firstly, it offers an energy-efficient method of computation, as not all neurons are required to be powered at any given time for robustness, as compared to classical computers. Secondly, it offers real-time responsiveness since computation is event-driven, resulting in lower latency and, thus, better user satisfaction and performance.

2.1.4. Impulse nature of information. Neurons communicate with one another via impulses or spikes, with the nature of the communication being dependent on both the timing of the impulse and the nature of the neuron firing the impulse, making communication sparse. This is very different in Classical Computing, where 'communication' among processes is synchronised by a global clock and largely consists of reading and writing to a central persistent memory, with 'communications' being dense in the Von Neumann Bottleneck. With information transfer being sparse and event-driven, energy usage is minimised, leding to energy efficiency.

The impulsive nature of the interaction between neurons makes computation in Neuromorphic computers computationally cheap and fast [2], making it suitable for Constrained Edge AI devices. This has been adopted by many chips, which include the Loihi 2 chip [1], which supports graded spikes for hybrid Artificial-Neural-Network/Spiking-Neural-Network models, and DYNAP, which utilises Address-Event Representation for spike routing for its analogue neurons.

2.1.5. Continual learning with on-device learning. On-device learning is an important characteristic of Neuromorphic Computers, whereby it is able to continuously adapt directly to its hardware [2]. There are multiple ways that learning can be performed on hardware, with the most prominent being Spike Timing Dependent Plasticity [8], [9], [10], whereby a connection between 2 neurons is strengthened or weakened based on the timing of their firing.

In the context of Federated Learning, on-device learning can reduce the Constrained Edge AI devices' reliance on the cloud for learning while also increasing the quality of learning by enabling real-time adaptation from real-time data. This can improve model accuracy and latency, thereafter increasing user satisfaction, which is important for edge devices. Furthermore, with on-device learning, user data privacy

and security are secured since learning is done locally. The AI models in these edge devices can even be trained to be personalised, too, with real-life examples like Akida being able to perform One-shot learning for personalised edge AI [6].

2.1.6. Sparsity. Unlike in Classical Computers, where computations are densely distributed over time, computation in Neuromorphic Computers is sparse [2], requiring only a small fraction of neurons to be active for sophisticated behaviour. Specifically, computation in Neuromorphic Computers is temporally sparse, meaning not many neurons are firing at any given time, and structurally sparse, whereby neurons are sparsely connected such that the flow of information is sparse. Temporal sparsity can provide the energy efficiency required for Constrained Edge AI devices. NeuroFlow (Grael One), for example, exploits frame similarity in video processing to skip redundant computations [6]. Structural sparsity can increase the quality of learning since learning for tasks is only done by neurons involved, thereafter providing a better user experience by allowing for a more accurate AI model and also increasing robustness as failure of some of neurons will not have an adverse effect on the overall functionality of Neuromorphic computers.

2.1.7. Analogue Computing. AI models in Classical Computers typically rely on numerical methods like gradient descent, which can be computationally expensive and sequential, thus making it unsuitable for Constrained Edge AI devices, whereby energy efficiency and response latency are important. In Neuromorphic Computers, however, analogue computations are performed [2] to run and train AI models. These computations manifest as oscillations of neural activity, where neurons fire asynchronously, mimicking biological neural dynamics. This approach is computationally cheap since neuron firing is inherently energy-efficient. In addition, analogue computing allows Neuromorphic Computers to incur lower data conversion overhead. Digital processors require continuous Analog-to-Digital (ADC) and Digital-to-Analog (DAC) conversions, which add energy and latency costs. Analog computing avoids this overhead by processing signals in their natural form.

One example of such modelling is BrainScaleS's RC circuits, which physically emulate the neuron's dynamic and perform analogous computations [6]. These analogue computations are also extremely responsive, as they naturally support parallelism and are fast [11], by supporting In-memory computing, which is discussed next.

2.1.8. In-memory computing. As mentioned above, computation in Neuromorphic Computers manifests as oscillations of firing between modules of neurons, and they are thus deterministic on the temporal states and connection configurations of neurons, instead of a well-defined set of instructions and memory (state). Essentially, information is encoded both temporally and within the states of neurons; these, in turn, are collocated with computations in Neuromorphic Computers [1], [2]. This avoids the overhead cost of reading and writing that Classical Computers incurs, making it suitable for Constrained Edge AI devices. This collocation of memory also suggests the redundancy of persistent memory, which allows Neuromorphic computers to be more compact and thus deployable for Constrained Edge AI devices. Memristors are one example of in-memory computing, and they are nanoscale devices that emulate synapses that can be commonly found in neuromorphic hardware like Loihi 1 & 2, IBM TrueNorth and many more [2]. Details of Memristors are elaborated on in the next section.

2.1.9. Hierachical organisation. Neurons within Neuromorphic Computers can be hierarchically organised, and this can provide better data processing by having complex data disentangled by each hierarchical level and having each level be specialised to deal with information at different complexities. Such a Hierarchical nature can also provide better AI explainability [2], as data processing can be compartmentalised and analysed at each hierarchical level. Such disentangling of data can also reduce data redundancy and, as such, increase latency and computational efficiency of the Neuromorphic Computer, making them suitable to be deployed in Constrained Edge AI devices.

2.2. Towards Neuromorphic Computing Hardware

In this section, the hardware implementation of a Neuromorphic Computer will be discussed in detail. The section starts by introducing the emulation of a neuron, followed by a memristor, a crucial electronic component for a Neuromorphic Chip, and its significance in Neuromorphic Computing. Lastly, it talks about the Intel Loihi 2 chip.

2.2.1. Modelling the Neuron. The Hodgkin-Huxley (H-H) Neuron Model is a system of differential equations that have been shown to be able to accurately model the behaviour of a brain neuron, specifically its spiking behaviour. It also has parameters that can be tuned to reflect different types of neurons.

While the H-H model has high biological fidelity, it is computationally expensive compared to a biological neuron. Therefore, simpler models have been introduced, like the Leaky Integrate-and-Fire (LIF) neuron model, the Quadratic Integrate-and-Fire model and most notably, the Izhikevich neuron, which attempts to capture the spiking behaviour of a neuron as much as possible while staying computationally efficient. These models trade off biological fidelity but can be considered sufficient to model neurons in some cases.

A model of a neuron allows one to mathematically simulate the behaviour of a brain, thus making neuromorphic computers a possibility. Additionally, models

that are energy efficient allow for computationally cheap models of neurons, which makes neuromorphic computing a viable option for constrained edge AI devices.

2.2.2. Memristor and the Memristive System. The memristor was a concept coined by Leon Chua in 1971, when he argued that there should be 4 fundamental passive circuit elements [1] so as to model the relationship between flux and charge for the sake of completeness [12]. By definition, the memristor is characterized by a relation of type Equation (1). It is said to be charge-controlled if said relation can be expressed as a single-valued function of the charge q.

$$g(\varphi, q) = 0, where\ \varphi := \ flux,\ q := charge \quad (1)$$

In a charge-controlled memristor, the voltage is given by

$$v(t) = M(q(t))i(t),\ where\ M(t) \equiv d\varphi(q)/dq \quad (2)$$

From these equations, one can see that the resistance of a memristor at any time t depends on the time integral of the memristor current from the initial starting time up to the current time t. In other words, at any time t, the current state of the memristor holds information about its past states, which it uses to determine its next states.

The concept of memristor has then been generalised to a much broader class of non-linear dynamical systems called the memristive system by Chua and Kang in 1976. The memristive system can be defined by Equations (3) and (4) and has been shown to be able to model the current-voltage characteristics of the Hodgkin-Huxley neuron model [13]. In the equation, w can be a set of state variables, and R and f can generally be explicit functions of time [13].

$$v = R(w, i)i \quad (3)$$

$$\frac{dw}{dt} = f(w, i) \quad (4)$$

In 2008, [14] presented a physical model of a two-terminal electrical device that is able to emulate the memristive system physically. Specifically, when variable $w(t)$ is within a certain range $[0 : D]$ – D being the thickness of a semiconductor in the physical model – the model behaves like a perfect memristor; and beyond this domain (but still bounded), it behaves like a memristive system. In this physical model, state variables $w(t)$ and $q(t)$ determine the total resistance across the physical memristor. To elaborate, the total resistance is determined by 2 different regions connected in series, whereby $w(t)$ abstractly represents one of the regions with a high concentration of dopants across the memristor's semiconductor, over a thickness D, which have low resistance R_{on}. $w(t)$, in turn, varies with the cumulative charge $q(t)$, and this is outlined in equation (5), where μ_v is the dopant mobility and R_{on} is the low-resistance state. Essentially, the resistance of the physical memristor is dependent on its state, which in turn is dependent on its input history.

$$\frac{dw}{dt} = \mu_v \frac{R_{\text{on}}}{D} i(t) \quad (5)$$

This tunable, history-dependent resistance enables neuromorphic circuits that emulate synaptic plasticity and neuronal dynamics. By integrating such devices into crossbar arrays, researchers achieve energy-efficient, adaptive systems capable of in-memory computing, key for next-generation edge AI and brain-inspired hardware.

2.2.3. Energy-Efficient Memory Processing. Within Classical Computing, a significant bottleneck and computation overhead arise from memory reads and writes, resulting in energy and latency issues in constrained edge devices that potentially deal with real-time data. However, with memristor, such concerns are sidestepped due to the temporal-dependent nature of memristor. Since memristors encode their past electrical history into their present state, they are suitable for in-memory computing, drastically increasing the energy efficiency of the device. This is especially relevant for constrained edge devices, which deal with temporal data and, in some cases, require immediate response. The ability of memristor to stay 'updated' to real-time data without reading/writing to any persistent memory makes it suitable for constrained edge devices.

Beyond in-memory computing, memristors enable asynchronous neuromorphic processing by physically emulating biological neural dynamics. Their analogue, non-volatile behaviour allows them to act as artificial synapses in spiking neural networks (SNNs), mirroring the brain's event-driven efficiency. In biological systems, neurons fire sparsely and on demand, minimizing energy use by avoiding constant activation. Similarly, memristor-based SNNs process data asynchronously, activating artificial neurons only when input spikes exceed a threshold. This sparse, event-driven paradigm is ideal for constrained edge devices handling unpredictable and intermittent workloads, such as intermittent sensor data or irregular visual inputs in AI cameras. For example, a security camera using memristive SNNs could process facial recognition triggers only when motion is detected, bypassing energy-draining continuous computation. By merging in-memory computing with brain-inspired asynchronous processing, memristors unlock ultra-low-power, latency-sensitive solutions for constrained edge AI applications.

2.2.4. The Intel Loihi 1 & 2. Intel's neuromorphic chips, Loihi 1 and Loihi 2, are designed to address the demands of decentralized, constrained edge devices

1. resistor, capacitor, inductor and memristor

through unique architectural innovations [4]. Loihi 1, fabricated on Intel's 14nm process, pioneered low-power, event-driven computation with 128,000 neurons and 128 million synapses per chip, consuming under 1W in applications like drone control and adaptive robotics. However, its reliance on proprietary interfaces and fragmented software (Nx SDK) limited its scalability and integration with conventional sensors. Loihi 2, built on Intel 4, significantly advances edge capabilities with 1 million neurons and 120 million synapses per chip alongside a 31 mm² die area for compact deployment. It introduces graded spikes (32-bit payloads), 3D mesh topologies, and local spike broadcast to reduce inter-chip bandwidth by 10x, enabling efficient decentralized networks. Hardware enhancements include standard interfaces (Ethernet, GPIO, SPI) for seamless sensor/actuator integration, a stackable 8-chip Kapoho Point system (4x4-inch form factor), and Sigma-Delta Neural Networks (SDNN) that improve inference speed and energy efficiency by 10x over Loihi 1. Software support is streamlined via the Lava framework, offering Python APIs, cross-platform simulation, and ROS/YARP compatibility, while flexible memory partitioning quadruples neuron density. With sub-200ns timesteps and microwatt-level power profiles, both chips excel in real-time edge applications—such as autonomous drones, smart sensors, and robotic control—where low latency, energy constraints, and decentralized operation are critical.

2.2.5. IBM TrueNorth. IBM's TrueNorth chip [3], developed under the DARPA SyNAPSE program and fabricated on a 28 nm low-power CMOS process, is a pioneering neuromorphic architecture designed for decentralized, power-constrained edge applications. The 4.3 cm² chip integrates 1 million programmable spiking neurons and 256 million synapses across 4,096 neurosynaptic cores arranged in a 2D mesh, collectively simulating essential neural structures such as soma, axon, dendrites, and synapses [15]. The architecture employs a purely event-driven design, characterized as Globally Asynchronous Locally Synchronous (GALS), wherein inter-core communication is asynchronous while computation within each core remains synchronous. This ensures energy-efficient spike routing with minimal latency, activating only during neural spike activity to significantly reduce dynamic power consumption [16].

Each neurosynaptic core integrates 256 programmable, non-linear, leaky integrate-and-fire (LIF) neurons, enabling deterministic, low-latency processing at 1 ms/tick intervals [15]. TrueNorth achieves a peak performance of 58 GSOPS and an energy efficiency of 400 GSOPS/W, with real-time workloads such as 30 fps HD video processing consuming just 65 mW [16]. Its hierarchical mesh network minimizes inter-chip traffic, while the chip's "glueless" tiling capability allows seamless integration into larger 2D or 3D arrays (e.g., 4×4-chip configurations) without additional interfacing components [17]. Compact edge platforms, such as the NS1e single-chip board, leverage this scalability for energy-efficient AI applications [15].

TrueNorth is optimized for vision and pattern recognition tasks, with applications spanning visual object recognition (DARPA Neovision2), robotic saliency mapping, and feature extraction using Haar and Local Binary Patterns (LBP) [16]. The architecture supports defect-tolerant redundancy and is programmable through the Corelet programming environment, which enables composable neural network configurations [17]. With its ultra-low power consumption, deterministic real-time operation, and robust scalability, TrueNorth is particularly suited for deployment in autonomous drones, smart sensors, and adaptive edge devices requiring high energy efficiency and low-latency processing [15].

2.2.6. SpiNNaker 2. SpiNNaker2 is a highly scalable, event-based neuromorphic system designed for energy-efficient machine learning and brain-inspired computing. It features 152 ARM Cortex M4F cores per chip, scalable to millions of cores in large-scale deployments, with 2GB DRAM per node and hardware accelerators for matrix multiplication, convolution, and random number generation [18]. Built in 22nm FDSOI technology, it employs dynamic voltage/frequency scaling (DVFS) and adaptive body biasing (ABB) to operate as low as 0.4V, drastically reducing power consumption while maintaining computational throughput [5]. The architecture leverages asynchronous, event-driven processing and sparse communication (Eg: spikes in SNN) to minimize data movement and idle power. For constrained edge devices, these features translate to ultra-low energy consumption (two orders of magnitude lower than CPUs in sparse training [18]) and sub-millisecond latency for real-time tasks like radar gesture recognition or biomedical signal processing. Its ability to execute batch-one inference with constant energy use, coupled with memory-efficient algorithms like deep rewiring (achieving 96.6% MNIST accuracy with 64 kB/core), makes it ideal for edge scenarios requiring low-power, decentralized computation. By combining energy-proportional operation, scalable parallelism, and sparse event-based communication, SpiNNaker2 bridges the gap between high-performance neuromorphic computing and resource-constrained edge applications.

2.2.7. Notable Mentions & Comparative Analysis (Hardware). The tables below outline the properties and specifications of the hardware mentioned above, and some notable mentions. From these tables, although BrainScaleS-2 is the better option, I have decided to go with Loihi 2 as it has a better community support and guides, in my opinion, which can encourage wide-scale adoption. It is also supported by Intel, which makes it more viable in terms of scalability and production.

, Hardware	Connectionism	Parallelism	Asynchrony	Impulse	Continual Learning	Sparsity	Analog	In-Memory
Loihi 2	Yes	Yes	Yes	Yes	Yes	Yes	No	Yes
IBM TrueNorth	Yes	Yes	Yes	Yes	No	Yes	No	No
SpiNNaker 2	Yes	Yes	Yes	Yes	No	Yes	No	No
BrainScaleS-2	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
DYNAP	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
Akida (BrainChip)	Yes	Yes	Yes	Yes	Yes	Yes	No	Yes
Innatera	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes

TABLE 1. COMPARISON OF NEUROMORPHIC HARDWARE CHARACTERISTICS

Hardware	Specifications	Chip Size
Loihi 2	128 neuromorphic cores, 1024 primitive neurons per core	31 mm² (Intel 4 process)
IBM TrueNorth	4096 cores, each with 256 neurons; 256x256 crossbar	430 mm² (28nm CMOS)
SpiNNaker 2	Over 1 million ARM processors	100 mm² per chip
BrainScaleS-2	512 adaptive integrate-and-fire neurons, 212k plastic synapses	20 mm²
DYNAP	NA	NA
Akida (BrainChip)	Ultra-low-power neuromorphic processor	8 mm² (28nm CMOS)
Innatera	NA	NA

TABLE 2. SPECIFICATIONS AND SIZE OF NEUROMORPHIC COMPUTING HARDWARE

2.3. Scaling Neuromorphic Computing

Neuromorphic Computers can function well together due to their decentralised and distributed nature. However, many challenges need to be addressed before Neuromorphic Computers can be integrated into decentralised constrained Edge Devices.

2.3.1. Standardising Software for Neuromorphic Computing. The implementation of Neuromorphic Computers at scale is still very underdeveloped and fragmented [2], [4], as methods used to implement Classical Computers at scale do not translate to Neuromorphic Computers. The majority of Neuromorphic systems often require low-level hardware details rather than high-level abstraction coding, which reduces community access [2], [4] and thus the adoption of Neuromorphic Computing. There is also a large diversity of neuromorphic hardware [2], [3], [19], all with different computational primitives and constraints across platforms. This can complicate the implementation of algorithms and models that are universal across neuromorphic devices. The fragmented ecosystem and lack of common standards also limit interoperability among neuromorphic devices and lead to difficulties in cross-compilation and seamless integration across different neuromorphic chips and systems. There is also a lack of standardised communication protocols with a variety of sensors and actuators, and the lack of integration tools for cloud scaling, deployment and system lifecycle management further limits the adoption of neuromorphic systems at the enterprise level. All these challenges are post-integration and deployment difficulties for neuromorphic computers in edge devices.

However, these challenges can be tackled with standardised software, which is capable of encapsulating low-level hardware details, such that it is a universal programming language across different neuromorphic devices or even non-neuromorphic devices to facilitate the adoption of neuromorphic computing. Standardising software can enable neuromorphic computers to tackle more complex tasks since irrelevant low-level hardware details can be encapsulated and ignored when developing algorithms and systems for complex tasks [2]. Fortunately, there have been developments that attempt to standardise software and unify the ecosystem for neuromorphic Computing.

2.3.2. Lava. One prominent and significant software is Lava, which is developed by Intel with Loihi 2 [4]. It is an open-source code, and one can easily access it via Github [20]. It is being managed by the Intel Neuromorphic Research Community with decent funding from Intel. This makes it a promising framework for Neuromorphic Computing, coupled with the influence Intel has in the Computing Industry. The motivation behind Lava is the unification of neuromorphic application development by providing abstractions for brain-inspired algorithms and cross-platform deployment. It addresses the fragmented ecosystem by standardising neuromorphic computing development tools across both neuromorphic chips and classical chips like CPUs and GPUs [20], [21], [22], addressing the barrier of entry resulting from lack of access to neuromorphic chips and cross-compilation difficulties across different neuromorphic chips. To elaborate, Lava's Magma compiler and runtime enable deployment across platforms by offering libraries for Python, and it provides third-party integration to tools like ROS, Tensorflow (lava-dl), and Nengo. It also simplifies benchmarking and comparison of neuromorphic algorithms [22], [23](9,46), allowing developers and engineers to better analyse the system for flaws or improvements. Lava also abstracts neurons and neuronal interactions (the atomic elements in Neuromorphic Computing) with *Processes* [21], [22],

which describes a functional group like a population of H-H neurons or a complete network. This abstraction provides behavioural flexibility by enabling a concept akin to *Interface* in C++, whereby a single *Process* can have multiple implementations (or *ProcessModel*) that are tailored to different hardware – neuromorphic or classical – or precision levels. This enables code reusability, thereafter improving software quality and lifecycle management resulting from reduced code redundancy. *Process* can also be nested, an abstract example being a neural network built from neurons (*sub-Process*), allowing better encapsulation and thus easing the design and implementation of complex systems. The bridge between classical and neuromorphic devices enables developers to prototype neuromorphic systems on pre-owned classical devices before deploying them on neuromorphic platforms, addressing enterprise investment concerns from moving towards neuromorphic devices [21], [22]. This can improve the adoption of neuromorphic computing by both community and enterprise and also improve integration with current edge devices, as it makes Neuromorphic Computing a practical solution with a promising software lifespan. Lava also comes with a library for deep learning (lava-dl), which can leverage present Artificial Neural Network models (ANN) by enabling ANN-to-SNN conversion. The integration of SNN can then sidestep training, which can be time and computationally intensive, allowing for faster and easier integration. In addition, the conversion method provided trains the SNN faster by dynamically estimating the equivalent ANN transfer function of a spiking layer with a piecewise linear model at regular epoch intervals and using the ANN equivalent network to train the original SNN.

However, Lava is still a work in progress, as it still struggles with optimising communication across hybrid architecture, which can present communication difficulties for decentralised systems and edge devices [24]

2.3.3. Notable Mentions & Comparative Analysis : Software.

PyNN : PyNN is a simulator-independent language for building neuronal network models [25], [26]. It is a work in progress that aims to provide a high level of abstraction while still allowing access to low-level information like states of neurons and synapses. It supports many hardware (Loihi 2, SpiNNaker 2 and much more) and simulators, and is currently being used by several large-scale simulation projects. It has decent documentation and is open-source, with decent progress being made. However, Lava is better maintained in my opinion, from looking through their respective GitHub repositories.

Nengo : Nengo Brain Maker is a Python package for building, testing and deploying neural networks [27]. It is highly extensible and flexible and can be used to implement networks for deep learning and complex tasks like motor controls. It has a well-maintained website that offers tutorials and guides in its documentation and a regulated forum where developers can discuss and offer advice & solutions to common problems. I chose Lava because it offers an easier alternative to quantize signal processing, which can offer significant improvement to the performance and latency of constrained edge devices, and also out of personal preference.

2.4. Learning in Neuromorphic Computers and Systems

Neuromorphic Computers integrate seamlessly into decentralised settings due to their inherent decentralised and distributed nature. Abstractly, a system of Neuromorphic Computers can be seen as mega-modules of neurons interacting via an external channel, like the cloud, to communicate information to one another. In the context of Federated Learning in Decentralised Edge devices, this information can be seen as 'knowledge' gained from local training data. To define the form that this 'knowledge' can take, we need to first understand how training is done on Neuromorphic Computers. Since training methods used on Classical Computers do not translate well into the Neuromorphic Paradigm, new learning methods have been developed to cater to learning in Neuromorphic Computers, and they are detailed below.

2.4.1. Plasticity. Before discussing training methods, it's crucial to first understand what drives the emergence of intelligence in neuromorphic computers, which is essentially driven by plasticity. Plasticity usually refers to the strength of connections between connecting neurons or their weights. As previously mentioned, Connectivity plays a big role in determining the behaviour of Neuromorphic Computers and is easily tangible, as they can be represented as a matrix of real numbers called the connectivity matrix. This matrix holds the weights of the connections between neurons, and by changing weights appropriately, the behaviour of Neuromorphic Computers can be tuned to match our demands. For example, Intel's Loihi 2 utilises programmable synaptic plasticity for on-chip learning [4].

2.4.2. Spiking Neural Network. It is also important to understand how neural networks are currently implemented on neuromorphic hardware, as they affect the choice of training method used. While it is possible to implement conventional ANN on neuromorphic computers, its architectures struggle to fully leverage the advantages of neuromorphic computing. Traditional ANNs are typically designed with feed-forward, fully connected layers and rely on synchronous, digital processing. When implemented on neuromorphic computers, these architectures fail to capitalize on the key benefits of neuromorphic computing, such as energy efficiency and low latency,

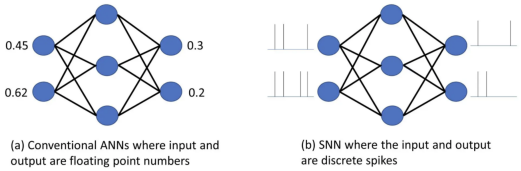

Figure 2. Figure shows the difference between SNN and ANN. The main difference lies in the type of input received by both networks. The figure is taken from [28]

which stem from properties like asynchrony and analogue computing. In contrast, SNNs are specifically designed to align with the architecture of neuromorphic chips. The same arguments can be made about running neuromorphic models on classical computers.

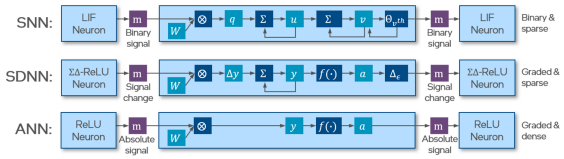

Figure 3. Figure depicts how information is passed through a 'neuron' in ANN, SDNN and SNN. The key feature of SDNN is signal change, which is the main form of information transfer. This reduces the amount of information transfer. The figure is taken from [29]

SNNs are biologically inspired networks that mimic the way neurons and synapses operate in the brain. They are used in neuromorphic chips such as IBM TrueNorth, Intel Loihi 2, and SpiNNaker 2, they enable real-time processing with minimal power consumption. Unlike traditional ANNs that rely on continuous activation functions, SNNs use discrete, event-driven spikes for asynchronous communication. For example, Loihi 2's Sigma-Delta neural network features a sigma-decoder in the dendrites (Input) and a delta-encoder in the Axon (Output). The delta encoder exhibits event-driven and asynchronous spiking behaviour by using differential encoding on incoming activations and only sending spikes when the encoded message magnitude exceeds its threshold. The sigma decoder reduces information processing redundancy by accumulating sparse event messages and maintaining them to the original value. In temporal sequences where data don't change as much, minimal activations occur between layers, simplifying connections between deeper layers by reducing them [29]. This spike-based processing not only enhances their biological fidelity but also significantly improves energy efficiency, making them well-suited for resource-constrained edge AI applications.

However, a major challenge in training SNNs arises from the non-differentiability of spike functions, which prevents the direct use of standard gradient-based learning methods commonly employed in ANNs. This necessitates the development of specialized learning techniques tailored for SNNs.

2.4.3. Surrogate Gradient Learning. Traditional backpropagation does not work on SNNs, as the spike function is typically non-differentiable as they are discrete spikes. Since traditional backpropagation requires gradients at each layer to be computable, this is not possible with SNNs, which have layers of discrete spike functions. There has been implementation which subverts the non-differentiable spike functions by approximating and replacing them with differentiable continuous functions like sigmoid during backward propagation. This is better known as Surrogate Gradient Learning (or, more generally, Spike-based quasi-backpropagation) [30] and can be used in Intel Loihi for local learning [31], [32]. While this method may sacrifice biological fidelity and the quality of learning from using approximations, it enables gradient-based optimisation for SNNs, which makes the adoption of SNNs easier by enabling the use of existing algorithms and software to implement learning.

To elaborate, [32] implements surrogate gradient learning on Loihi 1, which is fully on-chip. The Heaviside step function (6), typically used for binary spike activation, which is non-differentiable, is replaced during backward propagation with a truncated ReLU function (7). The derivative of the truncated ReLU function is approximated as a box function (8), enabling updates only when neuronal inputs are in the range $x \in [0, 1]$. This approach allows error gradients to propagate through the network during training, facilitating weight updates via standard backpropagation while maintaining compatibility with hardware constraints. By integrating this method with synfire-gated information routing on Intel's Loihi chip, the authors achieve competitive accuracy (95.7% on MNIST) and energy efficiency, demonstrating the first fully on-chip implementation of backpropagation in SNNs without external computational support. Substantial research interest is in this field of study.

$$f(x) = H(x - 0.5) \tag{6}$$

$$f_{surrogate}(x) = min(max(x, 0, 1) \tag{7}$$

$$f'(x) = H(x) - H(x - 1) \tag{8}$$

However, information within the temporal dynamics of spikes is not captured, and this may affect optimisation. There have been attempts to capture temporal dynamics by integrating Surrogate Gradient Learning with back.

2.4.4. Spike Timing Dependent Plasticity. Spike Timing Dependent Plasticity(STDP) is a form of learning method, or more specifically a temporally asymmetric form of Hebbian learning [9] induced by tight temporal correlations between the spikes of pre-and postsynaptic neurons [33]. STDP learns

under the assumption that the relative timing between the presynaptic and postsynaptic spiking determines the direction and the extent of synaptic changes, or rewiring, in the brain [34]. The equation and figure below give a general idea of STDP.

$$\Delta w = \begin{cases} A_+ e^{-\frac{\Delta t}{\tau_+}}, & \text{if } \Delta t > 0 \\ A_- e^{\frac{\Delta t}{\tau_-}}, & \text{if } \Delta t < 0 \end{cases}$$

Where:

$$\Delta w \text{ is the change in synaptic weight,} \tag{9}$$

$$\Delta t = t_{\text{post}} - t_{\text{pre}} \text{ (spike timing difference),} \tag{10}$$

$$A_+, A_- \text{ are learning rate parameters,} \tag{11}$$

$$\tau_+, \tau_- \text{ are time constants for potentiation and depression.} \tag{12}$$

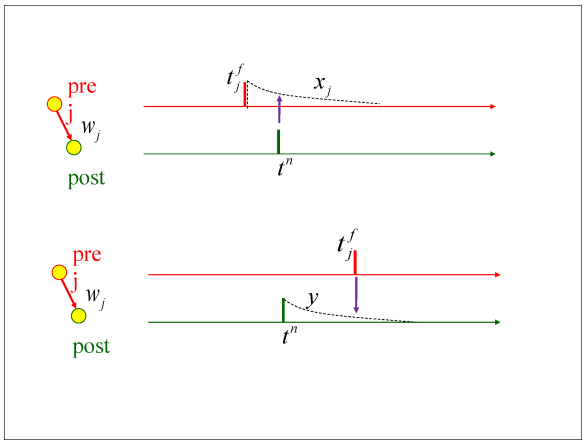

Figure 4. Figure shows the 2 scenarios depicted in the equation above. The top part depicts the case where pre fires before $post$, which strengthens the connection w_j by STDP. The below part depicts the case where pre fires after $post$, and thus, the connection w_j should be weakened. The figure is taken from [9], [10]

This classical STDP learning rule is implemented by Lava on Loihi chips with *STDPLoihi()* and *LearningDense* [35], where learning dense can be abstracted as connections with plasticity. In addition to this, Lava extends onto STDP with its Sum of the Product rule, which allows for customised learning rules to be integrated into the classical STDP learning rule, allowing for priors to be encoded within learning rules for faster convergence and making the STDP learning rule more flexible. This, in turn, is extended to the Three-Factor Learning Framework, which allows STDP to be extended to reinforcement learning tasks.

2.4.5. Sum of Product in Loihi & Lava. Loihi and Lava extend onto the classical STDP learning rule by allowing customised learning rules which manifest as equations relating the synaptic state variables (Weight W, Delay D and Tag T) to output synaptic target variables by implementing the Sum of Product Rule [36]. In relation to plasticity, Weight represents the strength of the connections or synaptic efficacy; Delay captures the temporal dependencies of the connections; and Tag is an additional synaptic variable that allows for constructing more complex learning dynamics. The equation below outlines the Sum of Product Rule.

$$Z \in \{W, D, T\} \tag{13}$$

$$dZ = \sum_{i=1}^{N_P} D_i \cdot \left[S_i \cdot \prod_{j=1}^{N_F^i} F_{i,j} \right] \tag{14}$$

The learning rule consists of a *sum* of N_P *products*. Each i'th product is composed of a dependency operator D_i, a scaling factor S_i, and a sub-product of N_F^i factors, with $F_{i,j}$ denoting the j'th factor of the current product.

(15)

For efficiency, trace and synaptic variable updates proceed in a learning epoch with a pre-defined length of t_{epoch} time steps. For each product and associated dependency operator D_i conditions, the product is on the presence of a pre or post-synaptic spike during the past epoch or evaluates the product unconditionally every other epoch. More specifically, it conditions a configurable number of pre/post-synaptic spikes per epoch or a configurable number of epochs, whereby it will be unconditionally executed after this specified period of epochs has passed. It also specifies the time steps within an epoch at which all trace variables in the corresponding product are evaluated. Factors within each component of the product can also assume different forms: Pre-synaptic spike with a configurable trace, Post-synaptic spike with a configurable trace, constant, synaptic state variables or their sign. To elaborate on traces, Loihi offers a set of 2 pre-synaptic traces x_1, x_2 and 3 post-synaptic traces $y_!, y_2, y_3$; and their dynamics are outlined below.

$$z \in \{x_1, x_2, y_1, y_2, y_3\} \tag{16}$$

$$z(t) = z(t_{k-1}) \cdot \exp\left(-\frac{t - t_{k-1}}{\tau^z}\right) + \xi^z \cdot \delta^z(t - t_k) \tag{17}$$

However, the Sum of Product rule has several limitations. Although the learning rules are highly customisable, they are constrained to a sum of product forms. Furthermore, trace and synaptic variable updates are computed with a slight delay at the end of each epoch, and although this will theoretically not have any impact as long as there is activity within the network (at least 1 neuron spiking per epoch), it can be a hard guarantee in such a complex dynamic network. This is even worse in the case of constrained edge devices, where malfunctions can happen, and the device can momentarily stop functioning.

2.4.6. Three-Factor Learning Framework. Classical STDP learning rules are typically purely unsupervised, and as such are only useful in classification tasks, making their use not extendable to other complex tasks which are hard to formulate or tackle as unsupervised learning tasks. Three-factor helps extend the classical STDP learning rule into reinforcement learning tasks by incorporating a third 'reward' factor that can be understood as a neuromodulatory signal that influences synaptic plasticity [37]. This allows learning to be moderated by reinforcement signals like cost, rewards and exploration, leading to more adaptive and behaviourally relevant learning. Furthermore, the three-factor learning framework has a high biological fidelity, as experiments have shown that neuromodulator does influence plasticity. For example, Dopamine release can determine whether STDP leads to long-term potentiation or depression [37].

Within Loihi, the Three-factor Learning framework is implemented as Reward-Modulated STDP (RM-STDP), and this is shown in the equation below. Specifically, synaptic weights, W is modulated as a function of the eligibility trace E and the reward term R [8]; and the weight updates Δw depends on $M(t)$, the level of the neuromodulator and $STDP(\Delta t)$ [37]. The neuromodulator level can be thought of as the equivalent of neurotransmitters like dopamines in the brain, such that it provides rewards/penalties to amplify/reverse plasticity between neurons. The eligibility traces track connections that have been involved in the firing of neurons, thereafter allowing delayed reward signals to modify their strength. This helps overcome delayed rewards, which is common in Reinforcement Learning.

$$\dot{W} = R \cdot E \tag{18}$$

$$\text{where} \quad \dot{E} = -\frac{E}{\tau_e} + STDP(\Delta t) \tag{19}$$

$$\text{and} \quad \Delta w = M(t) \cdot STDP(\Delta t) \tag{20}$$

Reinforcement Learning is mentioned here as this framework of relying on reward integrates into the framework of Reinforcement Learning (RL), making the learning process more intuitive as RL is a very natural way of learning in humans. This can make the integration of R-STDP a conceptually simpler process, thus reducing complexity in system design. It also enhances the usability of classical STDP, as classical STDP is purely unsupervised, and as such is only useful in classification tasks, making its use not extendable to other complex tasks which are hard to formulate as unsupervised learning tasks.

2.4.7. Evolutionary Algorithms. Evolutionary approaches can be used to train SNNs [30], and it is intuitive in the Neuromorphic Computing paradigm. Inspired by the learning performed in the brain, which consists of rewirings and changes in plasticity from its current state, Evolutionary algorithms learn by performing small changes to the solution at each iteration. More specifically, evolutionary algorithms start with a random pool of initial solutions called the population. From the population, each solution is evaluated against pre-defined metrics and assigned a score. This score is used to determine whether the solution 'survives' the iteration, whereby it will be used to 'reproduce' more solutions for the next population – done via a pre-defined combination function (a hyper-parameter) that combines 2 solutions such that the resulting solution can be effective. If it does not survive then selection, the solution is dropped from the population. This selection of solutions can be customised to drop out a certain proportion of the population at every iteration and is considered a hyper-parameter. To encourage exploration such that the search for a solution does not confine itself within the population, mutation can be introduced. A mutation is typically implemented by having the members of the newly repopulated population (after selection) generate a random value, and if it falls within the predefined mutation probability, small changes are made to the corresponding solution.

2.4.8. Reservoir Computing. Reservoir Computing is a machine learning framework designed for processing temporal data using recurrent neural networks (RNNs) with simplified training. It utilises a fixed, untrained 'reservoir' of interconnected neurons to non-linearly transform input signals into a high-dimensional state space, like a basis function in ANN [**?**], [7]. Learning is only performed in the output layer, which makes learning computationally tractable and, thus, ideal for constrained edge devices.

The reservoir is structured as a fixed, randomly connected recurrent network that processes temporal input data. Connections within the reservoir are typically sparse and randomised, to mimic the neuronal connection within the brain (eg functional MRI) which typically has a scale-free network structure comprising of 'leaf nodes' which are sparsely connected to a set of hub nodes called the rich club that is the 'central node' or densely connected nodes. This reservoir plays the role of dynamic memory within the system by capturing temporal dependencies in sequential data. This reservoir then connects to a linear output layer, which is trained to map the reservoir's state to the target output. Since the target outputs attempt to map to the ground truths, learning becomes a Regression problem, subverting the need for gradient optimisation as it has a closed-form solution. This also implies trained efficiency as a Regression problem.

Learning with Reservoir Computing can be affected by 3 factors. Firstly, the topology of the reservoir can affect memory retention, and they can be initially customised for specific tasks. Secondly, the encoding of the input data is also a design consideration, and data should be encoded such that the relationship (like the linearity of coordinates) between data is not lost.

Loihi and Lava support Reservoir Computing. Within Lava, in-library *Processes* like *Dense* and *lif* can be

used to create a Reservoir of LIF-Neurons.

2.5. Federated Learning

Federated learning is a form of distributed learning in a decentralised system. In the context of constrained edge devices, Under the current Federated Learning framework, clients collaboratively train a model under the orchestration of a central server, all while keeping training data decentralized [38]. This ensures data privacy and security among each participant, as it does not rely on the actual data to perform its training, meaning no local data is uploaded to the cloud. Instead, it relies on clients to inform the central system of the knowledge gained from the data, and this usually takes the form of a weight update corresponding to the local data [19]. Effectively, Federated learning embodies the principles of focused collection and data minimisation, allowing it to mitigate systemic privacy risks and costs resulting from traditional centralised Machine Learning.

2.5.1. Current Implementation. A typical workflow for Federated Learning can be broken down into 6 steps [38]. The first step is **Problem Identification**, followed by **Client Instrumentation**, where clients can be instrumented to store training data locally and maintain additional data or metadata. Then, in **Simulation Prototyping**, the model engineer may prototype model architecture and test learning hyperparameters in a Federated Learning simulation using a proxy dataset. With a satisfactory architecture, **model training**, **model evaluation**, and **Deployment** follow, much like a typical Machine learning model workflow, with differences arising in the details of the training and evaluation being done in a federated setting.

A typical federated training process is orchestrated by a server, and it constitutes 5 steps that are repeated by said server throughout the training process. In **Client Selection** , the server samples from a set of clients meeting eligibility requirements. These requirements are usually defined by the model engineer – for example: is the current device plugged in? Moving to the **Broadcast** step, selected clients download the current model weights and a training program from the server. Then, in **Client Computation**, each selected device locally computes an update to the model by executing the training program. Moving to the **Aggregation** step, the server collects an aggregate of the device updates and is the integration point for techniques like secure aggregation (for added privacy) and lossy compression of aggregates (for communication efficiency). Lastly, in **Model update**, the server locally updates the shared model based on the aggregated update computed from the clients that participated in the current round.

2.5.2. Deficiency of Current Federated Learning. The breakdown of the workflow and training of Federated Learning allows us to see the potential challenges that can arise and identify bottlenecks in the framework. Firstly, Federated learning struggles with task-complexity scalability as a complex task generally requires a computational-intensive and memory-intensive model that processes abstract complex data (ie images) required to solve the problem. Specifically, during Client Instrumentation, clients are required to store training data, and during Broadcast and Client Computation, the client not only needs to download a relatively large model but also needs to train the model, and these operations can be computationally and memory-intensive. The high latency caused by these devices can reduce user satisfaction with client devices, and even if requirements can be pre-fixed such that the concerns for users' experience are mitigated, it is still a sub-optimal approach as these requirements will only become more restrictive as task complexity increases, reducing the effectiveness of Federated Learning. This is especially true in a constrained edge device setting, where devices have limited computational resources. The computation capacity of edge devices is then a bottleneck for task-complexity scalability.

Furthermore, the Federated Learning framework is not fully decentralised (as learning is governed by the server) and fully distributed (as learning is mainly done by edge devices) as intended. In fully decentralised Federated Learning, communication with a central server to orchestrate learning is replaced with having Edge devices communicate with one another, such that the Peer-To-Peer learning method is performed, and learning is fully decentralised [38]. In the current implementations of Federated Learning, however, the server plays a crucial role in maintaining model integrity among clients (like in Client Selection, Aggregation and Model Update) by serving as a point where information is gathered and updated. This creates a vulnerable point where data and models can be compromised [1], [38]. Furthermore, since the clients are mostly responsible for performing the training program and communicating the information gained from local training data, the path of communication can be vulnerable to data interception. Even if data is encrypted or abstracted, information about data can still be inferred with appropriate analysis, making encryption or abstraction a sub-optimal approach to tackle communication vulnerability. As such, the current Federated framework is unable to fully realise the potential for data privacy and security that Federated Learning envision for a decentralised system.

The central server can potentially become a bottleneck for the decentralised system, as it plays a central role in orchestrating training among client devices. This presents scalability issues, as the number of clients a server can manage can be limited not by its processing power but by the channel of communication between them. This can have a significant adverse effect on the latency of constrained edge devices, as they might rely heavily on the computation of the central server, and this reliance can compromise their latency resulting from the communication channel's bottleneck, as channel usage tends to be sporadic yet

experiences bursts of heavy load due to the intrinsic temporal usage-dynamics of these devices, i.e City cameras during peak hours.

2.5.3. Challenges towards a fully-decentralised learning. Ideally, federated learning should be implemented such that there is no central server orchestrating learning, and this is instead replaced by a sparse network of connections between edge devices such that communication is sparse. This not only removes the bottleneck caused by the server but also improves latency as communication within devices is sparse. Learning should also be parallelised, such that it happens asynchronously so that the system remains fully decentralised by not being in total synchrony. This can help with robustness, as having limited access to edge devices will not affect the quality of learning [38] and achieve high energy efficiency from doing sparse and event-driven computations – which are commonplace for edge devices, where inputs can be sporadic. However, fully decentralised learning, or peer-to-peer learning, is not easily achievable, and there are many challenges ahead.

The first challenge can be classed under Algorithmic Challenges [38], which relates to the difficulty in designing algorithms that are robust for peer-to-peer learning. Due to the asynchronous nature and structural importance (network topology of clients) of a peer-to-peer learning set-up, the algorithms designed need to be robust to client dropouts and message loss. Asynchrony also risks model divergence due to asynchronous updates of the model. Secondly, algorithms have to account for the Non-IID(Identical and Independently Distributed) data across clients, as the nature of data can be linked to factors like geography for edge devices. However, it is difficult to do so as non-IID data can degrade model performance and complicate convergence guarantees. For example, Fully Decentralised SGD local updates mentioned in [39], [40] remain limited, relying on data assumptions or specific topologies, which causes its use in learning to be limited.

The second challenge lies with Privacy and Security [38] concerns in a fully decentralised setting. The lack of a central authority can make it difficult to detect attacks from malicious clients, potentially disrupting training. Furthermore, most current implementations of peer-to-peer learning rely on Blockchain, but the public visibility of the ledger can discourage participation. The trade-off for having Differential Privacy and not having a central server significantly reduces model utility as secure aggregation protocols are complex to implement with the absence of a central server.

Lastly, there exist practical and system challenges [38] that need to be tackled. Due to inherent differences in the accessibility of clients' devices for central servers, which can be affected by factors like device quality and geography, survival bias is induced as devices that are more accessible and active can skew the model towards their data. Furthermore, this can induce selection bias, as these devices can be over-represented in training too. Diverse implementations of hardware and software and different network conditions that clients function in can complicate code deployment and compatibility without having a central server acting as an interface for all client devices. In addition to this, diversity in data formats across devices can further complicate the implementation of peer-to-peer learning, and they may potentially have to be hard-coded to deal with differing formats. Monitoring and debugging also face privacy concerns, as logging of processes can be restrictive since such logging may violate or raise concern for the privacy of users. With the classical machine learning methods, which are computationally intensive, the lack of a central server can make it impractical for constrained edge devices, as they no longer have a central server to leverage for computation power.

2.5.4. Potential for Neuromorphic Computing. Taking a step back, one can argue that these challenges faced by the current frameworks of federated learning are inherent as it is implemented in a Classical Computing paradigm. Classical Computing is a central system, making it impossible to achieve a fully decentralised and distributed system for Federated Learning to be performed on. For example, since Classical Computers require a well-defined set of instructions and memory state for computation, it dictates that edge devices do the full computation of the training program. In an ideal distributed system, computations are well distributed, thus uplifting the bottleneck on task complexity by edge devices' computation capacity. Decentralised Classical Computers also dictate the need to maintain data integrity through data communications between clients and servers. This incurs overhead computation cost from read and write and demands a need for a point where data is centralised, thus creating a point of vulnerability. In an ideal decentralised and distributed system, computation information is distributed among the system, and information can only be recovered with the knowledge of the entire system. The distribution of information makes data private and secure, as the recovery of information is generally computationally implausible. The need to maintain data integrity also creates a strict temporal ordering of processes, which makes incorporating asynchrony and parallelisation into the system complex. Classical machine learning algorithms' performance is also highly dependent on the data that it is trained on, making it unsuitable to tackle non-IID data and vulnerable to data-induced biases.

Neuromorphic Computers, being one that is decentralised and distributed, then become ideal candidates for Federated Learning, as they can be easily extended to represent a fully decentralised and distributed system by treating each Neuromorphic Computer in the system as a module and their interaction as connections between modules. Firstly, computations within neuromorphic computers are sparse, event-driven and well-distributed among the system,

thus subverting energy concerns from deployment in constrained edge devices. Secondly, by extending the sparsity of neurons within the system to the sparsity of connection between neuromorphic computers, latency can be greatly improved, and models can be more personalised to their neighbourhood, resulting in the robustness of the model across different contexts. Distributed analogue computation also addresses the privacy concern of data uploading, as information within data is quickly dispersed across the system, and with the collocation of memory and state, leakage of user data is highly unlikely. Lastly, data-induced bias can be better addressed as learning within Neuromorphic Computers is more localised within their neighbourhood since learning is only performed on the sparse network of neurons involved, subverting challenges arising from imbalanced data affecting learning quality by skewing the model. All these make it easy to integrate the concept of fully decentralised Federated Learning, allowing for simpler designs with significantly less overhead costs from trying to simulate decentralised and distributed systems using centralised computers. However, as aforementioned, the general workflow and training process outlined above are largely meant for Classical Computers, and thus, there is a need to redefine the workflow and training process for one that accommodates Neuromorphic Computers.

2.6. Decentralised Constrained Edge AI devices

Decentralised Edge AI (DEA) devices describe a pool of devices that are either used by the user themselves or near the user. They differ from the usual cloud AI system, whereby only a central system hosts the AI model, and devices are reliant on the central server to make predictions. Instead, they can host an AI model and can both utilise the AI model for real-time predictions and train the AI model with real-time data. In the context of federated learning and decentralisation, a central entity is used to facilitate training among devices so that devices can benefit from their counterparts' training. In summary, DEA devices describe a decentralised system where computation is distributed among its population. However, in many cases, these devices have limited computational capacity relative to the data and AI model they are hosting, and as such, they are considered Decentralised Constrained Edge AI (DCEA) devices.

2.6.1. Reasons for DEA devices. The push for DEA devices arises from the fact that data coming from the edge of the cloud are increasing in both amount and complexity. This is largely due to the rise of the Internet of Things and advancements in technology, which have allowed devices to be deeply integrated into users' lives, allowing them to collect real-time data. This can present scalability problems for an AI cloud system, as data communication between edge devices and the cloud is potentially a bottleneck for the overall system [13]. This can lead to latency issues that result in low user satisfaction and functionality. For example, in the case of network cameras, which are commonplace in cities like London, DEA devices do not face this bottleneck as they are capable of making predictions locally, thus reducing latency. Furthermore, DEA devices ensure data privacy, enabling more effective use of data collected from a wide network of devices. [13] provides an example of finding a missing child with city cameras to support this argument. In typical cloud systems, if the missing child's photo is taken by 1 of the cameras, the data will not be sent to the cloud due to data privacy concerns. However, in the case of a DEA camera, it will be able to process the data locally, notify relevant authorities about the whereabouts of the child, train the model, and then delete the photo, ensuring the data privacy and safety of the child. The model updates are then propagated through the network of cameras, increasing their performance.

2.6.2. Challenges. The most prominent challenge lies in keeping the model pipeline computationally possible for the edge devices while still maintaining satisfactory performance. In practice, this manifests as an emphasis on keeping input dimensions low, making minimal data processing and using simple Neural Network models. For example, edge AI devices doing object detection that take images as input usually quantize the image to reduce input space and utilise the YOLO model for quick predictions. In complex tasks, the loss of information from over-reducing input data or the lack of a kernel function can make the task impossible to solve. Thus, the capability of DCEA is limited by the amount of computing in the local machine. In complex tasks, the loss of information from over-reducing input data or the lack of a kernel function can make the task impossible to solve. Thus, the capability of DCEA is limited by the amount of computing in the local machine.

While the advancement in technology has allowed Constrained Edge AI devices to scale well with task complexity, physical limits in chip design emphasise the need to look for other solutions to allow these devices to scale well in the future. One such solution can be to change the current framework of decentralised constrained Edge computing. Specifically, we want a framework that can better distribute computation, such that computation for a response is not fully done by an isolated entity but by the system itself. The final response can be abstractly divided into components, enabling any entity to reconstruct it without performing the entire computation. This approach allows edge devices to utilize the computational resources of cloud servers. However, such parallel processing of data in distributed systems incurs the overhead cost of maintaining data integrity among distributed devices in the Classical Computing paradigm. In the Classical Computing paradigm, computation and storage are fundamentally separated yet interdependent, and thus, every computation incurs the overhead cost of reading and writing the memory, which can be costly for

Constrained Edge Devices. Furthermore, the software for DCEA computers can be extremely complex and delicate, as a strict control flow of the system is required to ensure system integrity, and in many situations, parallel computing is not attempted due to the difficulty of writing software for the code, despite the efficiency it allows.

2.6.3. Potential for NC. Neuromorphic Computing does not face these challenges due to its unique architecture and operational principles. Firstly, it can achieve exceptional energy efficiency by integrating computation and state dynamics into a unified process. Unlike classical systems that separate processing and memory, neuromorphic architectures perform both tasks simultaneously, mimicking the brain's event-driven operation. This efficiency is further amplified by their reliance on sparse, binary communication (spikes or no spikes) between neuromorphic components, such as artificial neurons, which drastically reduces redundant data transmission and power consumption.

Furthermore, the absence of traditional persistent memory in these systems enhances data privacy and security. Instead of storing sensitive information in static memory, neuromorphic computers encode and distribute data transiently as spikes across their networks. In real-time applications—where inputs are continuous and processed instantaneously—this spiking activity is ephemeral and dispersed among countless interconnected components. As a result, reconstructing raw data from spike patterns becomes computationally intractable, especially in large-scale systems with dynamic, high-throughput inputs. This inherent obfuscation of data acts as a natural safeguard against data breaches, as there is no centralized "memory" to exploit.

Not only so, Neuromorphic computers' lack of traditional persistent memory inherently enhances their resilience and fault tolerance. By encoding information transiently as distributed spatiotemporal spike patterns across networks of artificial neurons, these systems avoid centralized points of failure. Synaptic plasticity enables self-healing, dynamically re-routing computations through redundant pathways if components degrade or malfunction. Their decentralized architecture eliminates bottlenecks, ensuring graceful performance degradation rather than catastrophic failure under stress. Additionally, event-driven, spike-based processing allows them to absorb real-time variability (Eg. noise, power fluctuations) without rigid timing dependencies. This robustness makes neuromorphic systems ideal for mission-critical decentralized AI applications—such as environmental monitoring or autonomous robotics—where reliability in unpredictable, harsh conditions is paramount.

Additionally, their decentralized architecture inherently avoids bottlenecks common to classical systems, such as parallelization challenges, while their reliance on localized spike-based processing minimizes vulnerabilities tied to data retention. These features position neuromorphic computing as a transformative paradigm for secure, energy-efficient DCEA devices.

3. Experiment

In this experiment, a task commonly faced by constrained-edge AI devices will be used to evaluate the performance of Neuromorphic Computing in such environments. A customised model and learning method will be selected for optimal performance and benchmarked against the State-of-the-Art (SOTA) open-sourced implementation. The comparison will be based on accuracy, latency (response time), and energy consumption. Additionally, the effectiveness of Federated Learning in accelerating learning on constrained-edge devices within a decentralised system will be assessed. Due to hardware and implementation constraints, the task will be simplified, and both training and testing will be conducted in a simulation.

3.1. Purpose

The experiment aims to show the effectiveness of current neuromorphic computers and architectures in constrained edge devices in a decentralised setting using Federated Learning. With the increased use of surveillance cameras in cities, I have decided to tackle a problem faced by them. Specifically, we will be looking at problems relating to Surveillance Camera Facial Recognition.

Surveillance Camera Facial Recognition is a common problem faced by city surveillance cameras, which are typically Constrained Edge AI devices. Surveillance with cameras has always been a cause of privacy concerns, yet they provided coverage over the city, which can aid police in the arrest and regulation of laws. For example, cameras can aid lost-and-found efforts with their wide coverage, potentially shortening the search time. This is crucial for the survivability/chances of finding the person. However, privacy concerns have prevented their effective use. In this task, a satisfactory model performance while assuring data privacy will be the most significant factor determining Neuromorphic computers' success and effectiveness, as these devices are deeply integrated into people's lifestyles. The model will also need to be robust to the physical differences of humans for fairness and social ethics.

3.2. Design

The experiment setup is as follows. To simulate the conditions of Constrained Edge devices, 4 Raspberry Pi 3Bs with 1 GB RAM and 8 GB storage will be used to run the models. They will be connected by simple HTTP connections and perform tasks under different circumstances to simulate differences faced by Constrained Edge devices (like non-IID of data, usage difference, and environmental difference). A new device is added at a later time to show the effectiveness of Federated Learning to facilitate training

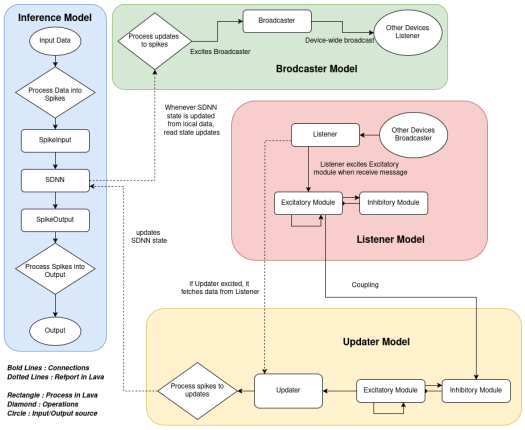

Figure 5. Overall Architecture

for joining devices, thus exhibiting scalability after deployment.

For this task, the model will be trained on this face-mask dataset available on Kaggle [41] in a Supervised Learning Setting. Given an image, the model will output whether the person in the image is wearing the mask properly. In practice, these images are received from a camera, but since access to Raspberry Pi cameras is limited, images will be sent via an HTTP connection to the model. To simulate the differences faced by surveillance cameras, images will be edited to reflect different weather conditions. Specifically, we will simulate pictures taken in the day (no edits), night (images are dimmed) and rain (images are blurred, Eg. due to raindrops on lenses). The model will train for a day and be evaluated on a dataset with even weather condition distribution. Accuracy and latency will be measured and benchmarked against available models.

After the initial models are trained, a new model is added to train together with the devices. To show the effectiveness of Federated Learning, the Accuracy over training time of the model will be plotted against a similar model trained individually on the same dataset.

3.3. Software

The project will be mainly based on Python 3.10+ using the Lava framework. Common supporting libraries will be used (NumPy ect). Memory wise, the libraries are able to be downloaded on a 8Gb Raspberry Pi with Linux installed (1GB). As such, the model is pretty lightweight, and easily deployable to edge devices with limited memory storage.

4. Implementations

The overall implementation can be seen in the diagram featuring 4 entities :

- Inference Model: Makes inference
- Broadcaster: Broadcast local 'learnt knowledge' to other devices
- Listener: Listens to 'learnt knowledge' from other devices
- Updater: Updates 'learnt knowledge' from other devices to local model

The code is available at https://github.com/BeanBois/iRecognise-, though it is still under-development.

4.1. Federated Learning on Neuromorphic Computers

4.1.1. Inference Model. The inference model contains 3 layers. Firstly, the Input Layer is responsible for collecting and processing the data, followed by the SNN Layer that does the inferencing, and finally the Output Layer, which is responsible for sending the output.

The Input Layer receives an image via a listening port from a central server. To process the image, the Input Layer first converts it to grayscale, then it sorts the image intensities based on the Hilbert Distance calculated using the index of pixels. The pixels are then binned into 100 bins, and using the mean values of each bin, spikes are generated with a Poisson distribution.

The spikes then pass through 2 Dense layers (that are not connected in series) to the SNN. The SNN receives the spike and passes through 2 layers of neurons, the first layer being made up of 10 modules of 200 excitatory neurons and the second layer being made up of 2 modules of 400 inhibitory neurons. In the first layer, the modules are split into 2 mega-modules (MM), each with 5 modules, and the 5 modules receive their spikes from the same Dense layer. Within each mega module, inter-module connections are generated with a probability of 0.2, with their weights randomly initialised. Within each module, 2000 randomly assigned connections are made. Mapping one mega-module to one inhibitory module in the second layer, for each inhibitory neuron in the inhibitory module, a random module is chosen from the corresponding mega-module, and from the chosen modules, 8 excitatory neurons are chosen to connect to. To simulate a Winner-Takes-All(WTA) situation appropriate for a Binary Classification Task, every inhibitory neuron in a given module connects to neurons in the same module and the opposing mega-module that it does not correspond to.

The excitatory modules are each connected to the Output Layer via an RSTDP Synapse layer. In implementation, the Output layer is not a Lava process, as it serves to aggregate the outputs of the learning dense layers from each Mega-module and makes a prediction by comparing these aggregated outputs. This is done by a simple comparison for simplicity.

4.1.2. Learning with Rewards. For learning, a reward will be 'sent' to the Inference model after the prediction is sent. I have used quotation marks as RM-STDP is not implemented yet for lava core libraries.

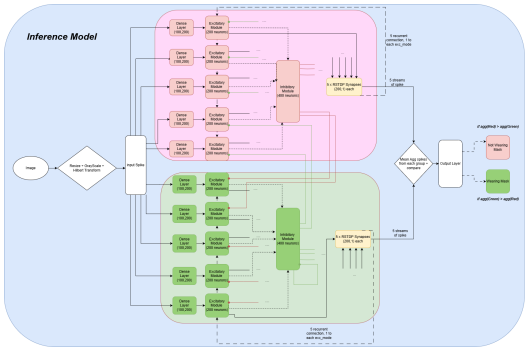

Figure 6. Inference Model Architecture. The Color of E-neurons and I-neurons represent the 'teams' of neurons in a WTA setup. Some edges have been discontinued for clarity. Colored lines represent inhibition by the inhibitory module with the corresponding color. Excitatory modules connect to similarly coloured RSDTDP Synapses.

However, in the case of Binary Classification and with our architecture, the application of a reward can be easily integrated into the usual STDP. As previously defined, know that the equations dictating STDP and RM-STDP are :

$$STDP : \Delta w = \begin{cases} A_+ e^{-\frac{\Delta t}{\tau_+}}, & \text{if } \Delta t > 0 \\ A_- e^{\frac{\Delta t}{\tau_-}}, & \text{if } \Delta t < 0 \end{cases}$$

$$\Delta w = M(t) \cdot STDP(\Delta t) \tag{21}$$

Since $M(t)$ in our case is simplified to $+1$ or -1, and that our architecture is built on a Winner-Take-All setup, by switching the signs of A_+ and A_- for each Learning Dense layer using the Classical STDP based on the predicted and actual label, reward can be induced into learning. Abstractly, we encourage one module of neurons and discourage the other for a particular label. This, however, is an extremely naive approach and may not scale well in complex tasks where relationship exists between class labels.

4.1.3. WTA setup. The WTA setup encourages neural competition among modules. In the context of our setup, abstracting the mega modules as 1 big excitatory pool of neurons and using the color scheme provided in the figure, the Red pool of E-neurons is put in competition against the Green pool of excitatory neurons due to the presence of the Inhibitory modules. Green E-neurons, when excited, excites Green I-neurons, which in turn surpresses Red E-neurons; and vice versa. This competitive setup is effective in a Binary Classification Task due to the task's either-or nature [42].

4.2. Federated Learning on Neuromorphic Computers

To facilitate Federated learning among the model, the inference model is connected to BroadCaster, Listener and Updater Model such that it can communicate and learn with other edge devices.

4.2.1. Broadcaster Model. To check for updates in the SNN, the Broadcaster has a RefPort that connects to the SNN model that records the weight changes in the RSTDP Synapse layer. After the 100th update, the broadcaster averages these updates and sends it to other devices with their device ID (randomly generated 16-bit int). Devices each receive the message via their Listener Model.

4.2.2. Listener Model. When the Listener model receives a Broadcast from a device, it first processes the device ID into spike (convert them into binary) and passes it through a Reservoir Computing module, where a decision of whether to update the model is made (the module needs to be trainable). If the update is approved by the Updater module, the weight changes are applied to the right dense layer (negative for negative, positive for positive prediction).

4.2.3. Updater Model. The updater module is the facilitator of Federated learning as it decides whether to accept the weights updates and is an attempt to further decentralise learning by removing the central facilitating entity. This, however, means that it learns a delayed reward since the impact of weights update might not be immediate, and as such, Reservoir Computing is used. More specifically, after the device ID is converted to binary spikes, it is passed into a reservoir of 200 randomly connected neurons via a Dense layer with randomly initialised sparse connections and weights. The output layer then does a weighted sum on the activation levels of each neuron in the Reservoir and applies a threshold of 0 on this sum. If the sum is above the threshold, the updater approves the weight updates.

To determine if a weight update was effective, the average accuracy in the last 50 predictions is compared against the average accuracy of the next 100 predictions. If there is an increase in accuracy, the output update output-RC layer connections with the reward. To deal with delayed rewards, random noise is injected into reward signals such that, over time, true rewards surface

5. Results & Evaluation

5.1. Results

During implementation, I faced hardware issues, namely, the model failed to run on the Raspberry Pi 3b. This may be because the Lava framework is not optimised for Classical Computers since parallelism within Lava is implemented as threads, and with many threads running, performance in Classical Computers suffers. This is especially true for constrained devices like Raspberry Pis, whose multi-threading capabilities are minimal. Due to a lack of access to computing devices (that I can use to run relatively intensive models for long) and connect with HTTP, I cannot continue with the decentralised setup. The output below shows the outputs from running *Inference Model* on a Raspberry Pi 3b:

```
python3 mainv3.py
[2782.513003] Out of memory: Killed
process 5459 (python3)
total-vm: 1386764kB ....
Killed
```

However, I have tried to run and train the inference model on my laptop. I have collected and plotted its accuracy over time (averaged over 10 minutes) and its response rate (calculated over 10 minutes), as shown above. Even with a laptop, the model was still computationally expensive, sometimes lagging out my computer. With the model not converging after more than 12 hours of training, I have decided not to continue with the training as I was not sure if the program was running robustly on my laptop, and I do not want to potentially spoil my computer. All in all, from this experiment, I think that Neuromorphic Computing is not yet ready to tackle complex tasks with limited computing resources. Below shows the plots of the Inference Model's Accuracies and Response Rate.

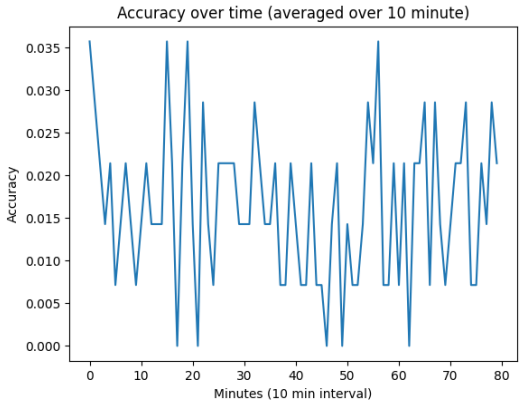

Figure 7. Figure shows the Inference model's ongoing learning accuracies over time, which are averaged over a minute for smoothness.

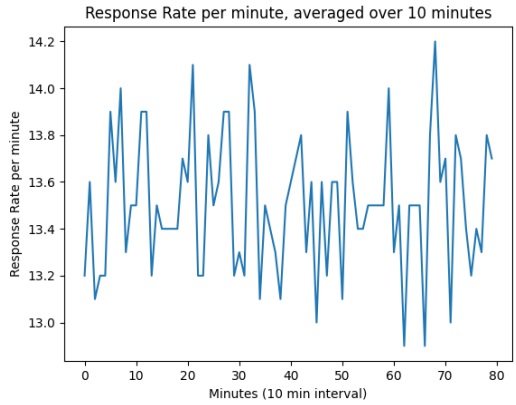

Figure 8. Figure shows the Inference model's response rate over time. This is measured in number of responses per minute

Looking at the accuracies, I suspect that my 'Reward Modulated Spike Timing Dependent Plasticity' did not work. My best guess for why this is the case is that due to the WTA setup, learning within the 2 groups are not cooperative, but competiting. With the recurrent connections being connected to each groups' excitatory modules during learning, both groups tries to inhibit each other during learning, which can lead to noise sypnatic weights updates. Additionally, the response rate is sub-optimal. Since it was ran on a M1 2020 Macbook pro with 16GB RAM, there is a higher expectation on its response rate, which it did not meet.

Additionally, the Lava framework was surprisingly complex to work with since Process and ProcessModels are relatively tightly coupled together. Since I had only learnt it for a month, I could not fully master it, and as such, I could not leverage its utilities. Debugging and looking for online community help was also difficult as it is a relatively new software, and as such, it has not been adopted by many. However, I believe that it is learnable with more time and experience and that the documentation and tutorials are sufficient in explaining the core concepts.

5.2. Evaluation

That being said, however, I still believe that Neuromorphic Computing is an up-and-coming technology that will revolutionise computing. Researchers with access to the neuromorphic devices have run experiments that show that they outperform classical artificial intelligence models in terms of energy usage [4], and in some tasks, they even match the state-of-the-art model performances [35], [36]. Given its maturity, it has been able to gain a significant amount of success, and with ongoing research, it will only get better.

Considerable research remains in the field, with ongoing studies focused on developing training methods and algorithms for neuromorphic computers [43]. Some of these efforts, such as Lava, are supported by open-source platforms, allowing developers to contribute and, most importantly, learn about it. With a growing number of developers familiar with it, there will be no shortage of developers for Neuromorphic Computers. This makes it practical for large-scale deployment.

Additionally, the commercial value of neuromorphic chips is highlighted by the involvement of technology giants like IBM and Intel, alongside smaller to medium-sized companies such as Innatura and BrainChip, which are actively researching and developing neuromorphic computing technologies. These companies are investing in neuromorphic chip research as a way to enhance machine learning, cognitive computing, and AI applications, showing the growing commercial interest and potential for this field. The access to funding also suggests that Neuromorphic Computers as a technology have a good survivalbility in the research community.

In future experimentations, I do want to explore federated learning in a robotic setting or cybersecurity setting. Specifically, in the robotic setting, I will want to experiment with federated learning between devices in different environments performing similar tasks. This can be, for example, 4-legged movement among robots in different terrains. An additional layer of complexity can be explored by introducing robots with different physical dimensions (different limb length) and observing if the decentralised models can converge to a general 4-legged-movement model. In the cyber security setting, I see its potential in being deployed in residential routers for better cyber security. I also want to explore if these devices can coordinate to detect large-scale cyber attacks.

6. Future Works

As aforementioned, considerable research remains in the field, and during my study in Neuromorphic Computing and Decentralised Systems, I have found concepts from Graphical Neural Network and Information Theory extremely relevant. Specifically, I hypothesise that methods from Graphical Neural Networks like Graph Aggregation, Dropouts and Node embedding and methods from Statistical Theory like Entropy and Generalised Measure of Information Transfer can apply to Spiking Neural Networks. Essentially, spikes between neurons can be abstracted as information transfer, and if we can measure their entropy via some node-embeddings, we can calculate the synergistic transfer, redundant transfer and unique transfer of information mentioned in the paper [29]. This can lead to robust sypnatic weights update – by ruling some as redundant and prioritising unique and synergistic transfer of information for example. Extending this to a Federated Learning context, using [29] Generalised Measure of Information Transfer as a metric can make learning robust by excluding noise in the weight updates. Extending even further, since SNN are essentially a graph, and every edge device has SNN with different graph topology, graph embedding methods can be used to better represent weight updates for other devices, since raw weight updates from a device might not translate as well to a different device. Overall, this has been a very eye-opening research, and I will want to continue reading on it.

References

[1] Furber S. Large-scale neuromorphic computing systems. Journal of Neural Engineering. 2016;13(5):051001.

[2] Kudithipudi D, Schuman C, Vineyard CM, et al. Neuromorphic computing at scale. Nature. 2025;637:801-12.

[3] Akopyan F, Sawada J, Cassidy A, Alvarez-Icaza R, Arthur J, Merolla P, et al. TrueNorth: Design and Tool Flow of a 65 mW 1 Million Neuron Programmable Neurosynaptic Chip. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems. 2015;34(10):1537-57.

[4] Corporation I. Neuromorphic Computing: Loihi 2; 2021. Accessed: 2025-02-04. Available from: https://download.intel.com/newsroom/2021/new-technologies/neuromorphic-computing-loihi-2-brief.pdf.

[5] Mayr C, Hoeppner S, Furber S. SpiNNaker 2: A 10 Million Core Processor System for Brain Simulation and Machine Learning; 2019. Available from: https://arxiv.org/abs/1911.02385.

[6] Ivanov D, Chezhegov A, Kiselev M, Grunin A, Larionov D. Neuromorphic Artificial Intelligence Systems. Frontiers in Neuroscience. 2022;16:959626. Available from: https://doi.org/10.3389/fnins.2022.959626.

[7] Dipert B. BrainChip Demonstration of Vision and Edge Learning Solutions for Event- and Frame-based Cameras — edge-ai-vision.com; 2023. [Accessed 18-02-2025]. https://www.edge-ai-vision.com/2023/08/brainchip-demonstration-of-vision-and-edge-learning-solutions-for-event-and-fram

[8] Team LD. Three Factor Learning with Lava;. Accessed: 2025-02-11. https://lava-nc.org/lava/notebooks/in_depth/three_factor_learning/tutorial01_Reward_Modulated_STDP.html.

[9] Hebb DO. Donald Olding Hebb. Scholarpedia. 2008;3(12). Accessed: 2025-02-13. Available from: http://www.scholarpedia.org/article/Donald_Olding_Hebb.

[10] Hebb DO. STDP-Fig2.png; 2025. [Accessed 13-02-2025]. http://www.scholarpedia.org/article/File:STDP-Fig2.png.

[11] Schmitt S, Klähn J, Bellec G, Grübl A, Güttler M, Hartel A, et al. Neuromorphic hardware in the loop: Training a deep spiking network on the BrainScaleS wafer-scale system. In: 2017 International Joint Conference on Neural Networks (IJCNN); 2017. p. 2227-34.

[12] Chua L. Memristor-The missing circuit element. IEEE transactions on circuit theory. 1971;18(5):507-19.

[13] Strukov D, Snider G, Stewart D, Williams RS. The missing memristor found. Nature. 2008;453(7191):80-3. Available from: https://doi.org/10.1038/nature06932.

[14] Tang G, Shah A, Michmizos KP. Spiking Neural Network on Neuromorphic Hardware for Energy-Efficient Unidimensional SLAM. arXiv preprint arXiv:230202758. 2023. Available from: https://arxiv.org/abs/2302.02758.

[15] Neuromorphic O. TrueNorth - IBM's Neuromorphic Hardware; 2025. Accessed: 2025-02-12. Available from: https://open-neuromorphic.org/neuromorphic-computing/hardware/truenorth-ibm/.

[16] Neuromorphic O. TrueNorth Deep Dive: IBM's Neuromorphic Chip Design; 2025. Accessed: 2025-02-12. Available from: https://open-neuromorphic.org/blog/truenorth-deep-dive-ibm-neuromorphic-chip-design/.

[17] Neuromorphic O. Scalability and Applications of TrueNorth; 2025. Accessed: 2025-02-12. Available from: https://open-neuromorphic.org/blog/truenorth-deep-dive-ibm-neuromorphic-chip-design/.

[18] Gonzalez HA, Huang J, Kelber F, Nazeer KK, Langer T, Liu C, et al.. SpiNNaker2: A Large-Scale Neuromorphic System for Event-Based and Asynchronous Machine Learning; 2024. Available from: https://arxiv.org/abs/2401.04491.

[19] Li L, Fan Y, Tse M, Lin KY. A review of applications in federated learning. Computers Industrial Engineering. 2020;149:106854. Available from: https://www.sciencedirect.com/science/article/pii/S0360835220305532.

[20] Team LD. Lava: A Software Framework for Neuromorphic Computing;. Accessed: 2025-02-11. https://github.com/lava-nc/lava.

[21] Team LD. Lava Architecture Overview;. Accessed: 2025-02-11. https://lava-nc.org/lava_architecture_overview.html.

[22] Team LD. Lava: A Software Framework for Neuromorphic Computing;. Accessed: 2025-02-11. https://lava-nc.org.

[23] Elfighi MMS, Mohammed YWA, Elghali OAE. Advancements and Challenges in Neuromorphic Computing: Bridging Neuroscience and Artificial Intelligence. International Journal for Research in Applied Science and Engineering Technology. 2025;13(2):123-30. Available from: https://www.ijraset.com/research-paper/advancements-and-challenges-in-neuromorphic-computing.

[24] Jyothi N, Rao AN, Velivela A, Shilpa S, Reddy MSRL, Sagar YA. In: Gunjan VK, Senatore S, Kumar A, editors. Neuromorphic Computing: Bridging the Gap Between Neuroscience and Artificial Intelligence. Singapore: Springer Nature Singapore; 2025. p. 81-92. Available from: https://doi.org/10.1007/978-981-97-8533-9_7.

[25] Ensemble N. PyNN - NeuralEnsemble — neuralensemble.org; 2025. [Accessed 22-02-2025]. https://neuralensemble.org/PyNN/.

[26] Davison AP, Brüderle D, Eppler JM, Kremkow J, Muller E, Pecevski D, et al. PyNN: a common interface for neuronal network simulators. Frontiers in Neuroinformatics. 2009;2. Available from: https://www.frontiersin.org/journals/neuroinformatics/articles/10.3389/neuro.11.011.2008.

[27] Nengo. Nengo — nengo.ai; 2025. [Accessed 22-02-2025]. https://www.nengo.ai.

[28] Sido E. Neuromorphic Devices in TinyML — Redeweb — redeweb.com; 2022. [Accessed 22-02-2025]. https://www.redeweb.com/en/Articles/neuromorphic-devices-in-tinyml/.

[29] Paul L Williams RDB. Generalized Measures of Information Transfer — arxiv.org; 2011. [Accessed 12-02-2025]. https://arxiv.org/abs/1102.1507.

[30] Schuman CD, Kulkarni SR, Parsa M, et al. Opportunities for neuromorphic computing algorithms and applications. Nature Computational Science. 2022;2:10-9.

[31] Kenneth Stewart SBSEN Garrick Orchard. On-chip Few-shot Learning with Surrogate Gradient Descent on a Neuromorphic Processor — arxiv.org; 2019. [Accessed 12-02-2025]. https://arxiv.org/abs/1910.04972.

[32] Alpha Renner AZLTAS Forrest Sheldon. The backpropagation algorithm implemented on spiking neuromorphic hardware - Nature Communications — nature.com; 2024. [Accessed 12-02-2025]. https://www.nature.com/articles/s41467-024-53827-9#citeas.

[33] Gerstner W, Kistler H. Spike-Timing-Dependent Plasticity. Scholarpedia. 2008;3(6). Accessed: 2025-02-13. Available from: http://www.scholarpedia.org/article/Spike-timing_dependent_plasticity.

[34] Bi, Poo. Synaptic Modifications in Cultured Hippocampal Neurons: Dependence on Spike Timing, Synaptic Strength, and Postsynaptic Cell Type — jneurosci.org; 1998. [Accessed 13-02-2025]. https://www.jneurosci.org/content/18/24/10464.

[35] Göltz J, Kriener L, Baumbach A, Billaudelle S, Breitwieser O, Cramer B, et al. Fast and energy-efficient neuromorphic deep learning with first-spike times. Nature Machine Intelligence. 2021 Sep;3(9):823–835. Available from: http://dx.doi.org/10.1038/s42256-021-00388-x.

[36] Lee JH, Delbruck T, Pfeiffer M. Training Deep Spiking Neural Networks Using Backpropagation. Frontiers in Neuroscience. 2016;10. Available from: https://www.frontiersin.org/journals/neuroscience/articles/10.3389/fnins.2016.00508.

[37] Frémaux N, Gerstner W. Neuromodulated Spike-Timing-Dependent Plasticity, and Theory of Three-Factor Learning Rules. Frontiers in Neural Circuits. 2016;9. Available from: https://www.frontiersin.org/journals/neural-circuits/articles/10.3389/fncir.2015.00085.

[38] Kairouz P, McMahan HB, Avent B, Bellet A, Bennis M, et al. Advances and Open Problems in Federated Learning. arXiv:191204977. 2019. ArXiv preprint. Available from: https://arxiv.org/abs/1912.04977.

[39] Wang J, Sahu A, Joshi G, Kar S. MATCHA: Speeding Up Decentralized SGD via Matching Decomposition Sampling. arXiv preprint. 2019 May;arXiv:1905.09435. Available from: https://arxiv.org/abs/1905.09435.

[40] Wang J, Joshi G. Cooperative SGD: A Unified Framework for the Design and Analysis of Communication-Efficient SGD Algorithms. arXiv preprint. 2018 August;arXiv:1808.07576. Available from: https://arxiv.org/abs/1808.07576.

[41] Jishan MA. Covid Face-Mask Monitoring Dataset — kaggle.com;. [Accessed 01-03-2025]. https://www.kaggle.com/datasets/jishan900/covid-facemask-monitoring-dataset.

[42] Lynch N, Musco C, Parter M. Winner-Take-All Computation in Spiking Neural Networks; 2019. Available from: https://arxiv.org/abs/1904.12591.

[43] Li S, Chao L. Tackling the Challenge of Small Sample Size: Effective Training by Integrating SNN and GNN. In: 2024 China Automation Congress (CAC); 2024. p. 1-4.

[44] Chen L, Zhang W, Xu Y. Edge AI: Intelligent IoT Systems. New York: Springer; 2020. Discusses the integration of AI into constrained edge devices and optimizing for performance and power consumption.

[45] Jo SH, Chang T, Ebong I, Bhadviya BB, Mazumder P, Lu W. Nanoscale Memristor Device as Synapse in Neuromorphic Systems. Nano Letters. 2010;10(4):1297-301. PMID: 20192230. Available from: https://doi.org/10.1021/nl904092h.

[46] Laskaridis S, Venieris SI, Almeida M, Leontiadis I, Lane ND. SPINN: synergistic progressive inference of neural networks over device and cloud. In: Proceedings of the 26th Annual International Conference on Mobile Computing and Networking. MobiCom '20. New York, NY, USA: Association for Computing Machinery; 2020. Available from: https://doi.org/10.1145/3372224.3419194.

[47] Lee U, Oh G, Kim S, Noh G, Choi B, Mashour G. Brain networks maintain a scale-free organization across consciousness, anesthesia, and recovery: evidence for adaptive reconfiguration. Anesthesiology. 2010;113(5):1081-91.

[48] Lim WYB, Ng JS, Xiong Z, Jin J, Zhang Y, Niyato D, et al. Decentralized Edge Intelligence: A Dynamic Resource Allocation Framework for Hierarchical Federated Learning. IEEE Transactions on Parallel and Distributed Systems. 2022;33(3):536-50.

[49] Lin H, Wang C, Deng Q, et al. Review on chaotic dynamics of memristive neuron and neural network. Nonlinear Dynamics. 2021;106:959-73. Available from: https://doi.org/10.1007/s11071-021-06853-x.

[50] McMahan B, Moore E, Ramage D, Hampson S, von Ehren B, B H. Communication-Efficient Learning of Deep Networks from Decentralized Data. Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS). 2017;54:1273-82. This paper introduces federated learning and explores decentralized methods for distributed model training while maintaining privacy. Available from: https://arxiv.org/abs/1602.05629.

[51] Shi W, Cao J, Zhang Q, Li Y, Xu L. Edge Computing: Vision and Challenges. IEEE Internet of Things Journal. 2016;3(5):637-46.

[52] Team LD. Lava Tutorial: End-to-End Tour;. Accessed: 2025-02-11. https://lava-nc.org/lava/notebooks/end_to_end/tutorial00_tour_through_lava.html.

[3] [32] [34] [44] [12] [4] [26] [7] [23] [25] [37] [1] [33] [18] [35] [9] [10] [6] [41] [45] [24] [38] [31] [2] [46] [36] [47] [19] [48] [49] [43] [42] [5] [50] [27] [17] [16] [15] [29] [46] [11] [30] [51] [28] [13] [14] [22] [20] [21] [52] [8] [40] [39]